Proyecto De Software Basado En Herramientas De Integración Continua

Entrega Proyecto 1 - Escenario 3

Edinson Hernandez

Jonathan Franco Santana

Alejandro Montoya Sandoval

Politécnico Grancolombiano

Integración Continua

John Olarte

10 de noviembre de 2026

Tabla de contenido

Introducción....1

Objetivo General....1

Objetivos específicos....2

Descripción Del Proyecto....2

Tecnologías empleadas....3

Arquitectura De La Solución....4

Comunicación Entre Los Contenedores....5

Persistencia Y Disponibilidad....5

Seguridad Y Gestión De Credenciales....6

Evidencias De Ejecución....7

Construcción de la imagen de la aplicación....7

Arranque de los servicios....8

Red Docker y servicios conectados....9

Prueba de comunicación entre Ghost y MySQL....9

Integración Continua....11

Conclusiones....12

Enlace A Repositorio De GitHub....13

Referencias....13

Introducción

La integración continua constituye una práctica fundamental dentro del desarrollo moderno de software, ya que permite automatizar procesos, mejorar la calidad de las aplicaciones y facilitar la colaboración entre los integrantes de un equipo de trabajo (Fowler, 2006). En ese contexto, las tecnologías de contenerización se han convertido en herramientas clave para garantizar entornos consistentes, portables y reproducibles durante todo el ciclo de vida del desarrollo.

En este documento se describe la primera entrega del proyecto GhostOps CI/CD, cuyo objetivo es implementar una solución basada en Docker con dos contenedores independientes capaces de comunicarse entre sí dentro de una misma arquitectura. El primero ejecuta la aplicación web Ghost y el segundo el sistema gestor de bases de datos MySQL. Esta implementación permite comprender los principios de la contenerización y su aplicación en escenarios reales de integración continua.

A lo largo del documento se presenta la configuración del repositorio en GitHub, la construcción de la imagen de la aplicación, los mecanismos de comunicación entre los servicios, la gestión de credenciales y la primera automatización mediante GitHub Actions. Asimismo, se incluyen las evidencias de ejecución real que validan el funcionamiento de la solución frente a los requisitos establecidos para la entrega.

Objetivo General

Implementar una infraestructura de software basada en contenedores Docker que permita ejecutar Ghost y MySQL como servicios independientes, demostrar la comunicación entre ambos con persistencia de datos y alojar el proyecto en un repositorio de GitHub con validación automatizada.

Objetivos específicos

- Configurar un repositorio en GitHub para el control de versiones del proyecto.
- Construir una imagen propia de la aplicación a partir de la imagen oficial de Ghost.
- Implementar un segundo servicio en un contenedor independiente con MySQL.
- Establecer la comunicación entre los contenedores mediante una red Docker definida por el usuario.
- Orquestar ambos servicios utilizando Docker Compose.
- Aplicar buenas prácticas de seguridad en la gestión de credenciales y en la exposición de puertos.
- Garantizar la persistencia de la información mediante volúmenes con nombre.
- Automatizar la validación del entorno mediante un flujo de trabajo de GitHub Actions.
- Documentar y evidenciar el correcto funcionamiento de la solución desarrollada.

Descripción Del Proyecto

GhostOps CI/CD es un proyecto académico que utiliza Ghost como plataforma de publicación web y MySQL como sistema gestor de bases de datos. Ghost es un gestor de contenidos de código abierto construido sobre Node.js que requiere de forma obligatoria un motor de base de datos externo para almacenar su información, lo que lo convierte en un caso de estudio adecuado para una arquitectura de dos contenedores.

La infraestructura se implementa mediante contenedores Docker independientes, administrados con Docker Compose y conectados a través de una red privada. Esta separación responde al principio de responsabilidad única aplicado a la infraestructura: cada contenedor

ejecuta un solo servicio, puede escalarse o reemplazarse de forma autónoma y declara explícitamente sus dependencias.

La aplicación no se ejecuta directamente desde la imagen oficial, sino desde una imagen propia del proyecto construida mediante un Dockerfile que parte de ghost:6.0.0-alpine y declara de forma explícita el directorio de trabajo, el puerto expuesto y el comando de inicio. De este modo, la construcción de la aplicación queda bajo el control del repositorio y puede ser verificada en cada cambio por el flujo de integración continua.

El proyecto evolucionará hacia un pipeline completo de integración y entrega continua con Jenkins, Travis CI y Codeship en las entregas posteriores. El control de versiones se encuentra alojado en un repositorio de GitHub, en el que cada integrante trabaja sobre su propia rama y la rama principal se conserva como referencia histórica.

Tecnologías empleadas

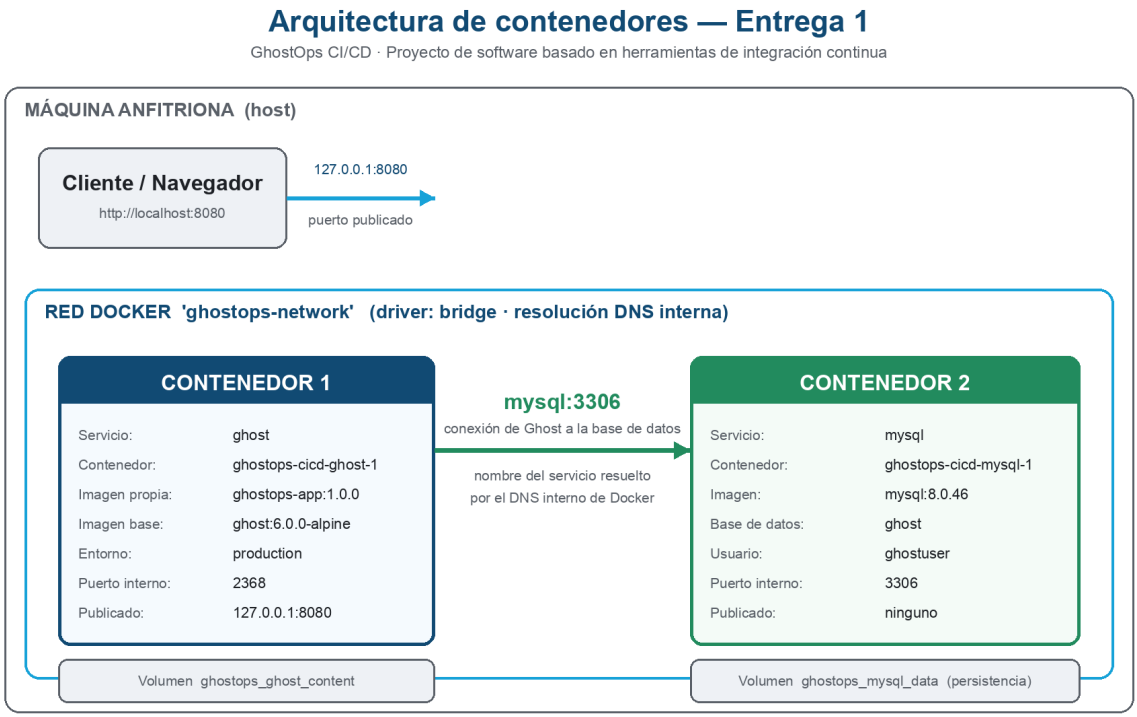
- Ghost CMS 6, construido desde la imagen base ghost:6.0.0-alpine, como aplicación web de publicación.
- MySQL 8.0.46, como sistema gestor de bases de datos relacional.
- Docker Engine y Docker Compose V2, para la contenerización y la orquestación.
- Git y GitHub, para el control de versiones del proyecto.
- GitHub Actions, para la validación automatizada del entorno en cada cambio.
- Windows 11 con Docker Desktop y PowerShell, como entorno de ejecución local.
- Jenkins, Travis CI y Codeship, previstos para las etapas posteriores.

Arquitectura De La Solución

La plataforma está compuesta por dos contenedores que se ejecutan de forma aislada y que se comunican exclusivamente a través de una red interna de Docker creada para el proyecto. El primer contenedor ejecuta la aplicación Ghost y es el único que publica un puerto hacia la máquina anfitriona. El segundo ejecuta el motor de base de datos y no publica ningún puerto, por lo que únicamente resulta alcanzable desde dentro de la red del proyecto.

Figura 1

Arquitectura de los dos contenedores del proyecto.



Nota. Elaboración propia a partir del archivo compose.yaml del repositorio.

Conviene señalar que la publicación del puerto de la aplicación se realiza sobre la interfaz de bucle local, mediante la declaración 127.0.0.1:8080:2368. De esta manera el servicio queda disponible únicamente desde la propia máquina y no se expone a la red exterior, lo cual constituye una buena práctica de seguridad en entornos de desarrollo.

Ambas imágenes se declaran con etiquetas de versión fijada y no con la etiqueta latest. Esta decisión garantiza que la infraestructura sea reproducible: cualquier integrante del equipo, o el servidor de integración continua, obtiene exactamente las mismas versiones de Ghost y de MySQL que se emplearon durante las pruebas documentadas en esta entrega.

Comunicación Entre Los Contenedores

Los servicios se conectan mediante una red Docker de tipo bridge denominada ghostops-network. Al tratarse de una red definida por el usuario, Docker habilita en ella un servidor de nombres interno que resuelve el nombre de cada servicio a la dirección que el contenedor posee dentro de la red (Docker Inc., 2026).

Por esta razón, Ghost utiliza el nombre del servicio mysql como dirección de conexión, declarado en la variable de entorno database__connection__host. No se emplea una dirección IP fija, puesto que esta cambia cada vez que el contenedor se recrea, ni tampoco localhost, ya que dentro de un contenedor ese nombre apunta al propio contenedor y no a los demás servicios de la red.

La comprobación de esta comunicación no se limita a la configuración. Ghost crea su propio esquema de datos en MySQL durante el primer arranque y almacena allí cada publicación del sitio. En consecuencia, consultar una publicación directamente en la base de datos y obtenerla después desde el navegador demuestra que el flujo de escritura y de lectura entre ambos contenedores se encuentra operativo, tal como se evidencia más adelante.

Persistencia Y Disponibilidad

El sistema de archivos de un contenedor es efímero, de modo que la información se perdería al eliminarlo o recrearlo. Para evitarlo se definen dos volúmenes con nombre: ghostops_mysql_data, que conserva los datos de MySQL, y ghostops_ghost_content, que

conserva el contenido de Ghost. Gracias a ellos, la orden docker compose down detiene los servicios sin destruir la información del sitio.

En cuanto a la disponibilidad, MySQL incorpora una comprobación de salud mediante la directiva healthcheck, que ejecuta periódicamente la orden mysqladmin ping. Docker Compose utiliza esta comprobación junto con la condición depends_on para esperar a que la base de datos se encuentre efectivamente disponible antes de iniciar Ghost. Sin esta precaución, la aplicación intentaría conectarse antes de que el motor estuviera listo y el arranque fallaría de forma intermitente.

La aplicación declara igualmente su propia comprobación de salud, que consulta su página principal mediante una petición HTTP interna. De este modo, la orden docker compose up --wait no devuelve el control hasta que ambos servicios informan del estado Healthy, lo que permite encadenar pruebas automatizadas con la certeza de que el entorno está listo.

Seguridad Y Gestión De Credenciales

Las contraseñas de la base de datos no se escriben en el archivo de orquestación. Estas se declaran como variables de entorno obligatorias con la sintaxis de interpolación de Docker Compose, de manera que el entorno no puede levantarse si no se han definido previamente. El archivo que las contiene se encuentra excluido tanto del control de versiones como del contexto de construcción de la imagen.

- El archivo .env está ignorado por Git y por .dockerignore, de modo que las credenciales reales nunca llegan al repositorio.
- El archivo .env.example documenta las variables necesarias sin contener valores reales.

- El contenedor de MySQL no publica su puerto en la máquina anfitriona; solo es accesible desde la red interna del proyecto.
- La aplicación se publica únicamente sobre la interfaz 127.0.0.1, por lo que no queda expuesta a la red local.
- El flujo de integración continua emplea credenciales temporales de prueba, nunca las credenciales personales del equipo.

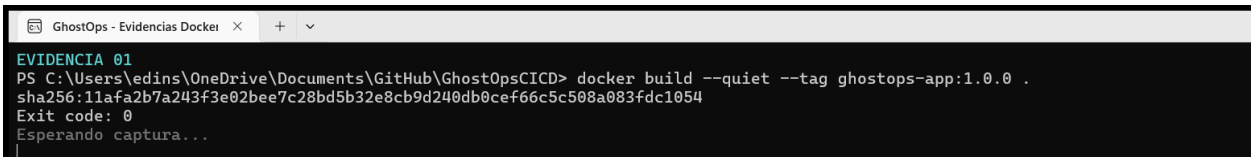
Evidencias De Ejecución

A continuación se presentan las evidencias obtenidas durante la ejecución real del entorno en la máquina de uno de los integrantes del equipo. Las salidas que se muestran corresponden a los comandos efectivamente ejecutados desde PowerShell y se reproducen sin modificación alguna.

Construcción de la imagen de la aplicación

Figura 2

Construcción de la imagen propia de la aplicación.



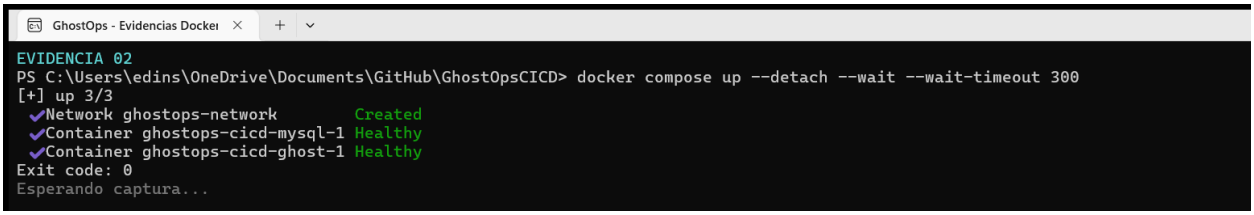
Nota. Captura de pantalla propia tomada durante la ejecución local del proyecto el 7 de noviembre de 2026.

La orden docker build generó la imagen ghostops-app:1.0.0 a partir del Dockerfile del repositorio y devolvió el identificador de la imagen resultante junto con un código de salida igual a cero, lo que confirma que la construcción finalizó sin errores. Este paso es el que permite que la aplicación se ejecute desde una imagen versionada por el propio proyecto y no directamente desde la imagen oficial.

Arranque de los servicios

Figura 3

Inicio de los servicios con Docker Compose.

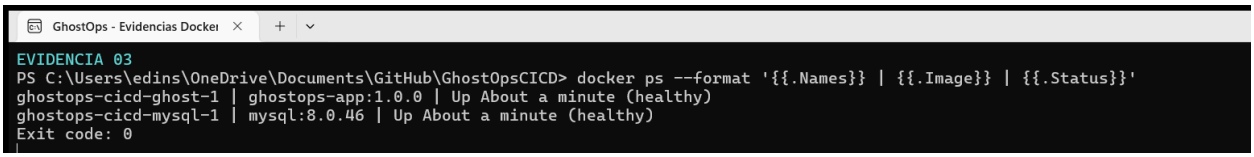


Nota. Captura de pantalla propia tomada durante la ejecución local del proyecto el 7 de noviembre de 2026.

La orden docker compose up --detach --wait creó la red ghostops-network e inició los dos contenedores del proyecto. El indicador up 3/3 y el estado Healthy de ambos servicios demuestran que Docker Compose esperó a que las comprobaciones de salud se cumplieran antes de devolver el control, de modo que el entorno quedó verificadamente operativo y no solamente iniciado.

Figura 4

Estado de los contenedores en ejecución.



Nota. Captura de pantalla propia tomada durante la ejecución local del proyecto el 7 de noviembre de 2026.

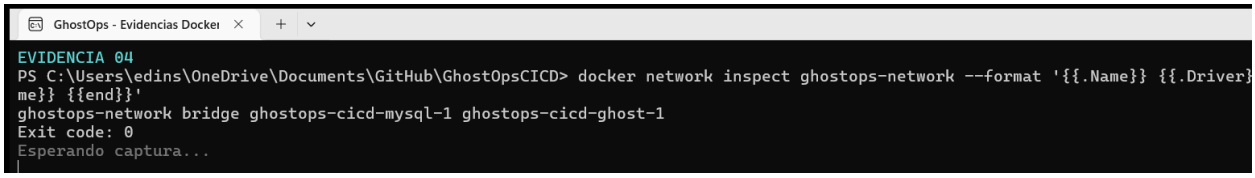
La consulta del estado confirma que los dos contenedores se ejecutan de forma simultánea e independiente: ghostops-cicd-ghost-1, a partir de la imagen propia ghostops-app:1.0.0, y ghostops-cicd-mysql-1, a partir de la imagen mysql:8.0.46. Ambos aparecen en

estado healthy, lo que indica que sus respectivas comprobaciones de salud continúan superándose durante la ejecución.

Red Docker y servicios conectados

Figura 5

Inspección de la red del proyecto y de los contenedores conectados a ella.



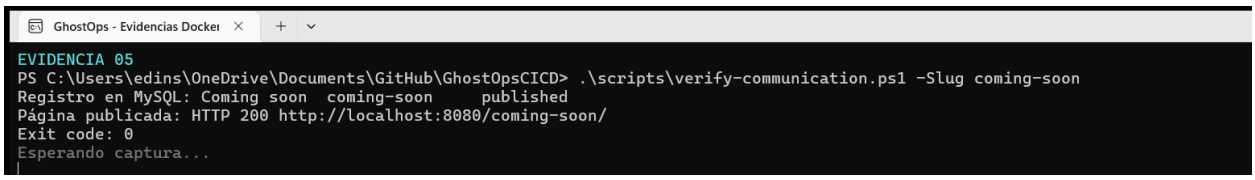
Nota. Captura de pantalla propia tomada durante la ejecución local del proyecto el 7 de noviembre de 2026.

La inspección de ghostops-network muestra que se trata de una red con controlador bridge y que los dos contenedores del proyecto se encuentran conectados a ella. Esta es la condición que habilita la resolución interna de nombres: al compartir una red definida por el usuario, cada contenedor puede alcanzar al otro empleando el nombre del servicio como si fuera un nombre de dominio.

Prueba de comunicación entre Ghost y MySQL

Figura 6

Verificación de la comunicación entre la aplicación y la base de datos.

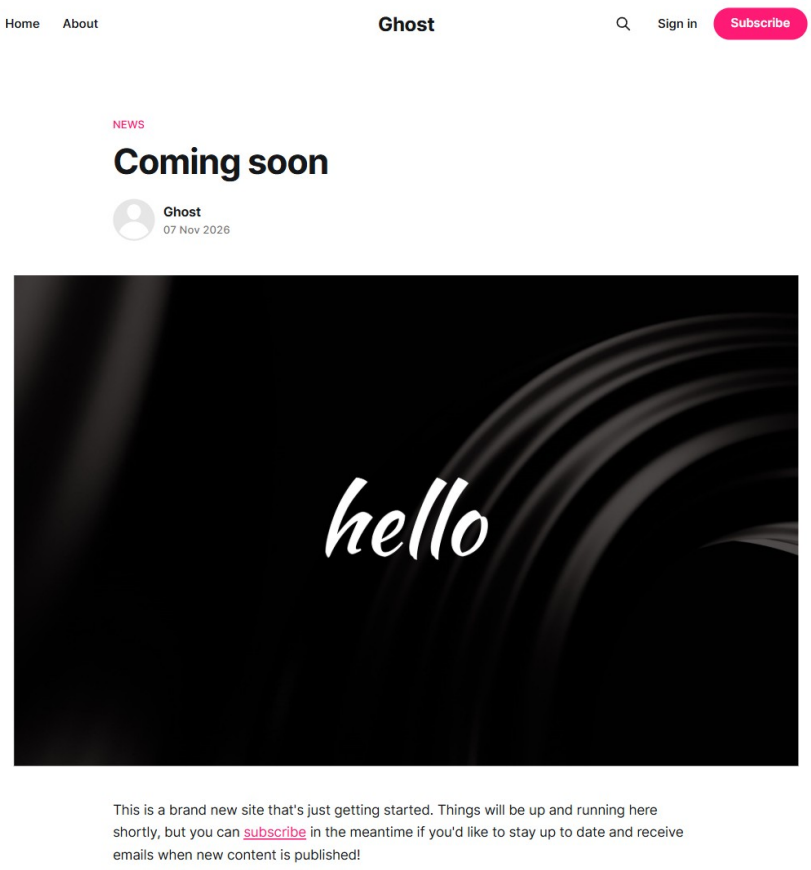


Nota. Captura de pantalla propia tomada durante la ejecución local del proyecto el 7 de noviembre de 2026.

El script verify-communication.ps1 realiza dos comprobaciones encadenadas. En primer lugar consulta directamente el contenedor de MySQL y recupera el registro de la publicación coming-soon, que figura con estado published. En segundo lugar solicita esa misma dirección a la aplicación y obtiene una respuesta HTTP 200. Dado que el registro consultado en la base de datos no fue insertado manualmente, sino creado por Ghost desde el otro contenedor, esta evidencia demuestra el flujo completo de escritura y lectura entre ambos servicios.

Figura 7

Publicación servida por la aplicación en el navegador.



Nota. Captura de pantalla propia tomada durante la ejecución local del proyecto el 7 de noviembre de 2026.

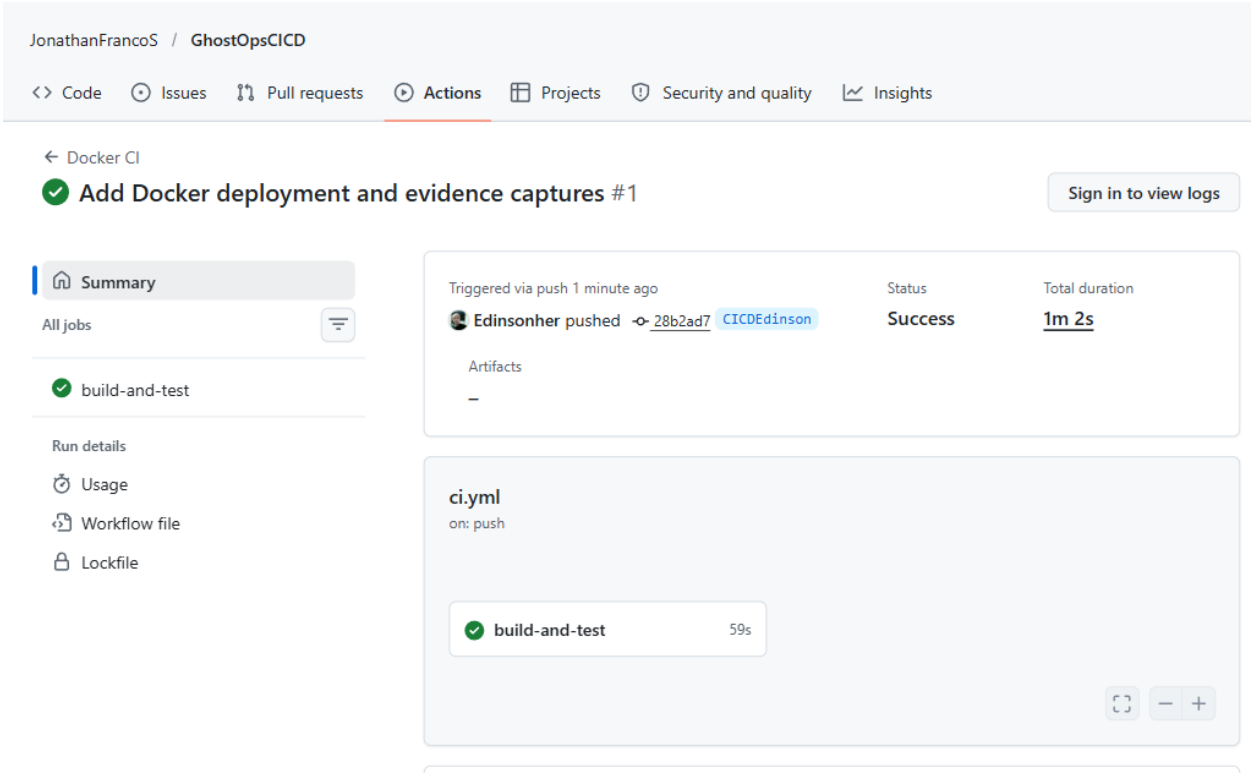
La misma publicación se visualiza desde el navegador en la dirección publicada por el contenedor de la aplicación. Puesto que Ghost no puede iniciarse ni servir contenido sin una conexión válida a su base de datos, el hecho de que la página se muestre correctamente confirma que la comunicación entre los dos contenedores se encuentra establecida.

Integración Continua

El repositorio incorpora un flujo de trabajo de GitHub Actions que se ejecuta automáticamente con cada envío de cambios y con cada solicitud de incorporación. El flujo valida la configuración de Docker Compose, construye la imagen de la aplicación, inicia ambos servicios esperando sus comprobaciones de salud, verifica que la aplicación responda por HTTP y comprueba que Ghost haya inicializado la tabla posts dentro de MySQL.

Figura 8

Ejecución exitosa del flujo de integración continua en GitHub Actions.



Nota. Captura de la ejecución número 1 del flujo Docker CI en el repositorio del proyecto, finalizada con estado Success. Recuperado de https://github.com/JonathanFrancoS/GhostOpsCICD/actions/runs/42404274686

La ejecución finalizó con estado Success en una duración de un minuto y dos segundos. Los avisos que acompañan al resultado corresponden a notificaciones informativas de mantenimiento de la plataforma y no afectan al resultado de la validación. Esta automatización constituye el primer paso del pipeline de integración continua previsto para las entregas siguientes, ya que garantiza que ningún cambio pueda incorporarse al proyecto sin que el entorno completo haya sido construido y verificado.

Conclusiones

La implementación permitió establecer una primera infraestructura basada en contenedores, separando la aplicación web de su sistema gestor de bases de datos. Se cumplió el objetivo de la entrega, puesto que el proyecto se encuentra alojado en un repositorio de GitHub y se construyeron dos contenedores Docker que se comunican entre sí, lo cual quedó demostrado mediante evidencias de ejecución real y no únicamente a través de la configuración declarada.

La comunicación entre contenedores no depende de direcciones IP sino del nombre del servicio, gracias a la resolución de nombres que Docker habilita en las redes definidas por el usuario. Este mecanismo es precisamente lo que hace que el entorno sea reproducible en cualquier máquina, condición indispensable para los procesos de integración continua.

Se comprobó igualmente que no basta con que un contenedor se encuentre iniciado para que su servicio esté disponible. La combinación de la comprobación de salud con la condición depends_on resultó indispensable para que el arranque fuera fiable, requisito necesario para que un flujo automatizado no falle de forma intermitente.

Finalmente, esta arquitectura constituye la base sobre la cual se incorporarán las herramientas de integración y entrega continua previstas para las entregas siguientes, dado que la contenerización elimina la diferencia entre el entorno de desarrollo y el del servidor de integración (Humble & Farley, 2010).

Enlace A Repositorio De GitHub

El código fuente completo del proyecto, junto con el archivo de orquestación, el flujo de integración continua, la documentación y las evidencias, se encuentra versionado y disponible en la siguiente dirección:

https://github.com/JonathanFrancoS/GhostOpsCICD

El equipo trabaja mediante ramas individuales, de modo que cada contribución queda registrada de forma independiente y la rama principal se conserva como referencia histórica de la evolución del proyecto.

Referencias

Docker Inc. (2026). *Docker documentation*. https://docs.docker.com/

Fowler, M. (2006). *Continuous integration*. https://martinfowler.com/articles/continuousIntegration.html

Ghost Foundation. (2026). *Ghost documentation*. https://ghost.org/docs/

GitHub Inc. (2026). *GitHub Actions documentation*. https://docs.github.com/actions

Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.

Oracle Corporation. (2026). *MySQL 8.0 reference manual*. https://dev.mysql.com/doc/refman/8.0/en/

Politécnico Grancolombiano. (2026). *Proyecto de software basado en herramientas de integración continua* [Guía del proyecto grupal]. Módulo Énfasis Profesional I.